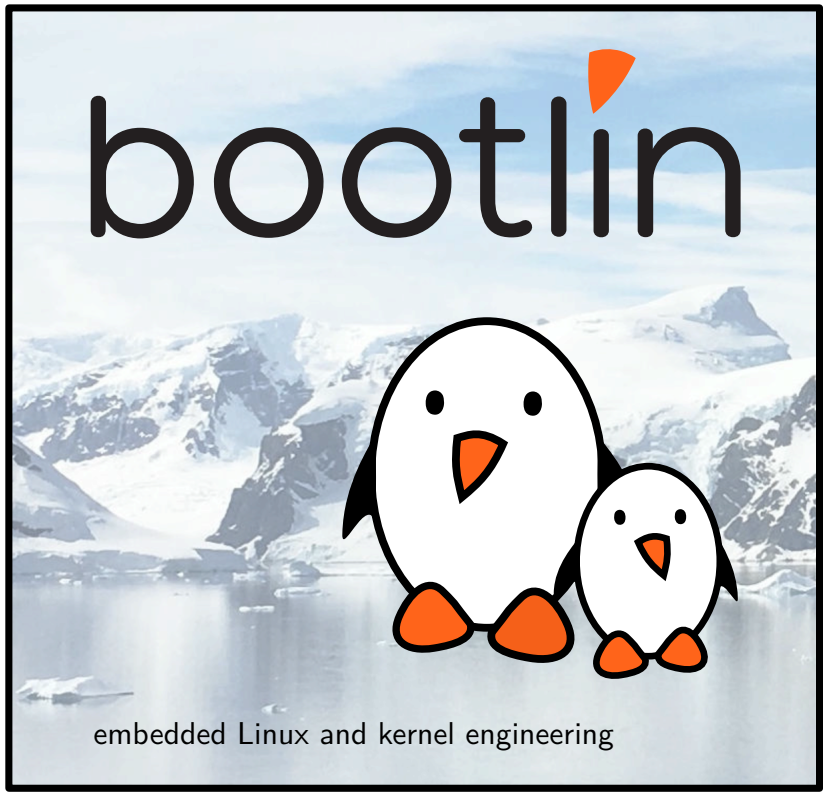
Using Yocto Project - basics



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Environment setup



Environment setup

- All Yocto source files are left unchanged when building a custom image.
- Specific configuration files and build repositories are stored in a separate build directory.
- A script, `oe-init-build-env` , is provided to set up the build directory and the environment variables (needed to be able to use the `bitbake` command for example).



oe-init-build-env

- Modifies the environment: has to be sourced!
- Adds environment variables, used by the build engine.
- Allows you to use commands provided in openembedded-core.
- `source ./oe-init-build-env [builddir]`
- Sets up a basic build directory, named `builddir` if it is not found. If not provided, the default name is `build`.



The initial `build/` directory

- The `oe-init-build-env` script creates the `build` directory with only one subdirectory in it:
  - **conf/** Configuration files. Image specific and layer configuration



Exported environment variables

BUILDDIR Absolute path of the build directory.

PATH Contains the directories where executable programs are located. Absolute paths to `scripts/` and `bitbake/bin/` are prepended.



Available commands

bitbake	The main build engine command. Used to perform tasks on available recipes (download, configure, compile...).
bitbake-*	Various specific commands related to the BitBake build engine.



Configuring the build system



The `build/conf/` directory

- The `conf/` directory in the `build` one holds two mandatory build-specific configuration files:

  bblayers.conf

  Explicitly list the layers to use.

  local.conf

  Set up the configuration variables relative to the current user for the build.
  Configuration variables can be overridden there.

- Additional optional configuration files can be used:

  site.conf

  Similar to `local.conf` but intended to be used for site-specific settings, such as network mirrors and CPU/memory resource usage limits.

  toolcfg.conf

  Used by tools to store build configuration, it should not be modified manually.



Configuring the build

The `conf/local.conf` configuration file holds local user configuration variables:

- BB_NUMBER_THREADS: How many tasks BitBake should perform in parallel. Defaults to the number of CPU threads on the system.
- PARALLEL_MAKE: How many processes should be used when compiling. Defaults to the number of CPU threads on the system.
- MACHINE: The machine the target is built for, e.g. `beaglebone`.



Building an image



Compilation

- The compilation is handled by the BitBake *build engine*.
- Usage: `bitbake [options] [recipename/target ...]`
- To build a target: `bitbake [target]`
- Building a minimal image: `bitbake core-image-minimal`
  - This will run a full build for the selected target.
- The `oe-init-build-env` script lists some more example targets



The build/ directory after the build 1/2

conf/	Configuration files, as before, not touched by the build.
downloads/	Downloaded upstream tarballs of the recipes used in the builds.
sstate-cache/	Shared state cache. Used by all builds.
tmp/	Holds all the build system outputs.



The build/ directory after the build 2/2

tmp/work/

Set of specific work directories, split by architecture. They are used to unpack, configure and build the packages. Contains the patched sources, generated objects and logs.

tmp/sysroots/

Shared libraries and headers used to compile applications for the target but also for the host.

tmp/deploy/

Final output of the build.

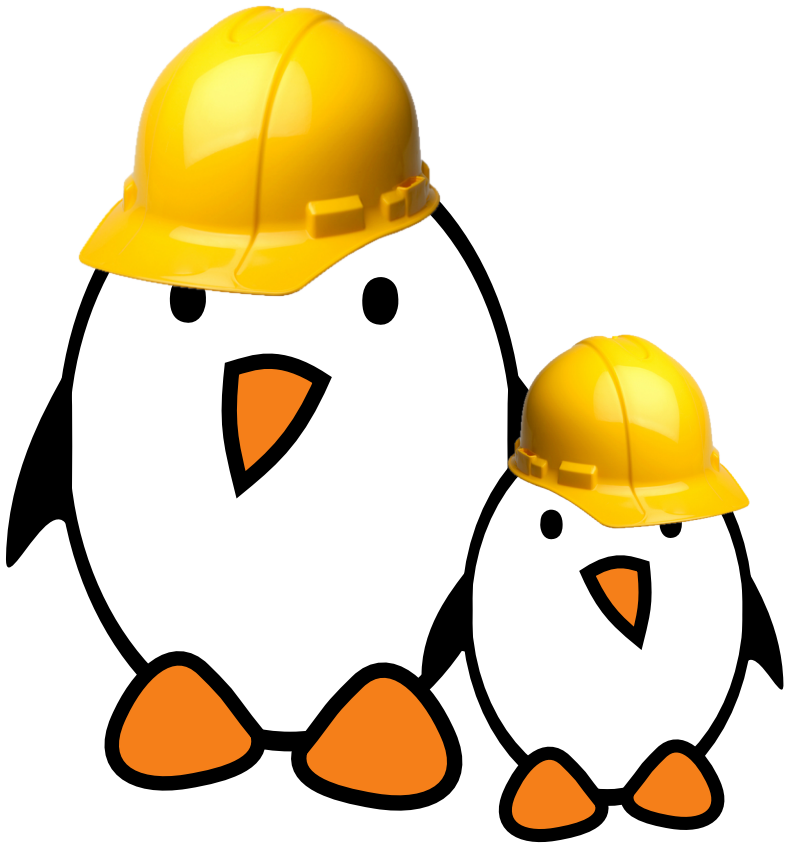
tmp/deploy/images/

Contains the complete images built by the OpenEmbedded build system. These images are used to flash the target.

tmp/buildstats/

Build statistics for all packages built (CPU usage, elapsed time, host, timestamps...).

Practical lab - First Yocto build



- Download the sources
- Set up the environment
- Configure the build
- Build an image

Using Yocto Project - advanced usage



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Advanced build usage and configuration

- Variable operators and overrides.
- Select package variants.
- Manually add packages to the generated image.
- Run specific tasks with BitBake.



A little reminder

- A *Recipe* describes how to fetch, configure, compile and install a software component (application, library, ...).
- These tasks can be run independently (if their dependencies are met).
- All the available packages in the project layer are not selected by default to be built and included in the images.
- Some recipes may provide the same functionality, e.g. OpenSSH and Dropbear.



Variables



Overview

- The OpenEmbedded build system uses configuration *variables* to hold information.
- Variable *names* are in upper-case by convention, e.g. `CONF_VERSION`
- Variable *values* are strings
- To make configuration easier, it is possible to prepend, append or define these variables in a conditional way.
- Variables defined in **Configuration Files** have a **global** scope
  - Files ending in `.conf`
- Variables defined in **Recipes** have a **local** scope
  - Files ending in `.bb`, `.bbappend` and `.bbclass`
- Recipes can also access the global scope



Operators: basic assignment

- `VAR = "value"` simply assigns a value
- Re-assigning overwrites variable value

```
VAR = "this"
VAR = "that"
```

Result: `VAR = "that"`

- Newlines need to be escaped (this does not apply to functions)

```
LIST = "this \
        and that"
```

- Variable assignments can contain expansion of other variables

```
COLOUR = "blue"
SKY = "the sky is ${COLOUR}"
```



Operators: immediate expansion

- With =, expansion happens when the variable is used
- Use := for immediate expansion

```
COLOUR = "blue"
SKY = "the sky is ${COLOUR}"
COLOUR = "grey"
PHRASE = "Look, ${SKY}"
```

Result:

```
PHRASE = "Look, the sky is grey"
```

```
COLOUR  = "blue"
SKY := "the sky is ${COLOUR}"
COLOUR  = "grey"
PHRASE = "Look, ${SKY}"
```

Result:

```
PHRASE = "Look, the sky is blue"
```

- Normal expansion is correct in most cases. Only use := when really needed.



Operators: appending and prepending

- Variable values can be modified by composition:

  `+=` append (with space)
  `=+` prepend (with space)
  `.=` append (without space)
  `=.` prepend (without space)



Operators: default and weak default values

- The `?=` operator assigns a value only if the variable has not been assigned when the statement is parsed

```
COLOUR = "blue"
COLOUR ?= "unknown"
```

Result: `COLOUR = "blue"`

```
COLOUR ?= "unknown"
```

Result: `COLOUR = "unknown"`

- The `??=` operator assigns a value only if the variable has not been assigned when the statement is parsed, not even using a `?=` operator
- The BitBake documentation explains the differences in details.



Operators caveats

- The operators apply their effect during parsing
- Example:

```
VAR ?= "a"
VAR += "b"
```

Result: `VAR = "a b"`

```
VAR += "b"
VAR ?= "a"
```

Result: `VAR = " b"`

- The parsing order of files is difficult to predict, no assumption should be made about it.
- To avoid the problem, avoid using `+=`, `=+`, `.=` and `=.` in `$BUILDDIR/conf/local.conf`. Always use overrides (see following slides).



bitbake-getvar

- `bitbake-getvar` can be used to understand and debug how variables are assigned
- Lists each configuration file touching the variable, the pre-expansion value and the final value
- `bitbake-getvar <VARIABLE>`: dump the variable value in the **global** scope
- `bitbake-getvar -r <recipe> <VARIABLE>`: dump the variable value in the **local** scope of `recipe`

```
$ bitbake-getvar DEPLOY_DIR
NOTE: Starting bitbake server...
#
# $DEPLOY_DIR [2 operations]
#   set? /home/user/yocto-labs/openembedded-core/meta/conf/bitbake.conf:440
#     "${TMPDIR}/deploy"
#   set /home/user/yocto-labs/openembedded-core/meta/conf/documentation.conf:137
#     [doc] "Points to the general area that the OpenEmbedded build system uses to place images, [...]"
# pre-expansion value:
#   "${TMPDIR}/deploy"
DEPLOY_DIR="/home/user/yocto-labs/build/tmp/deploy"
$
```



Overrides

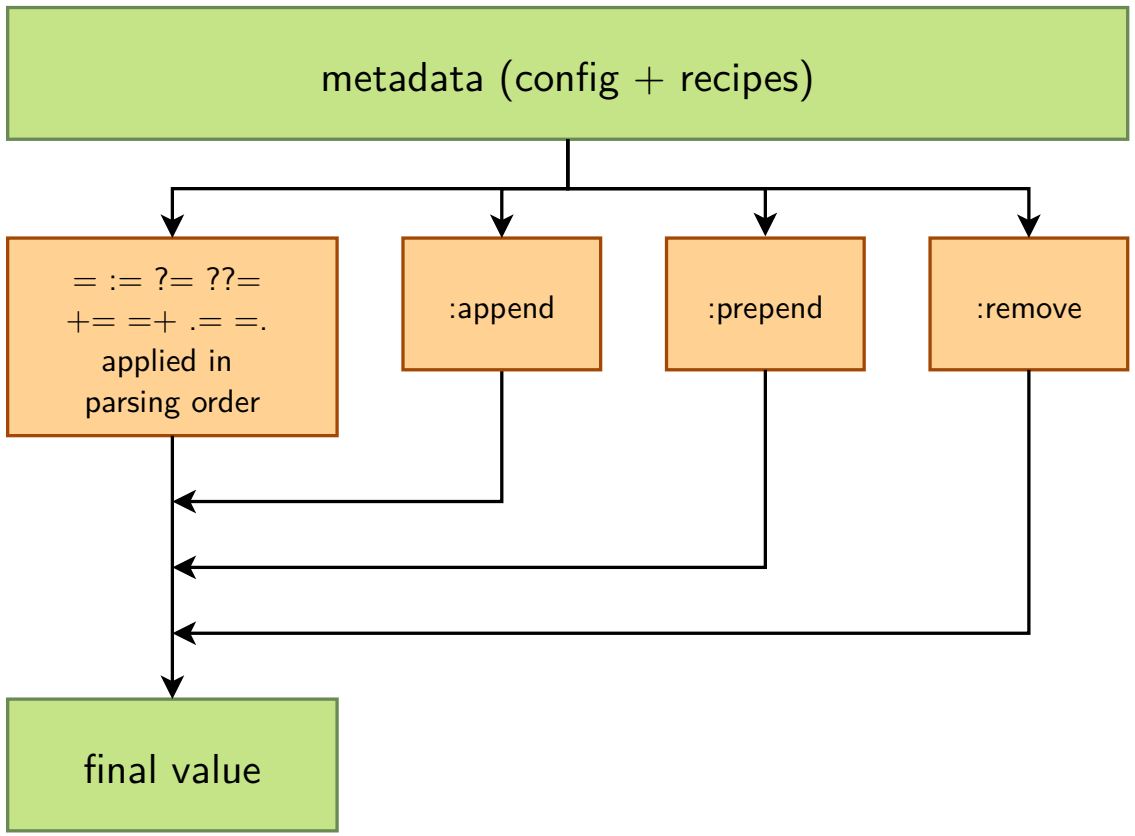
- Bitbake **overrides** allow appending, prepending or modifying a variable at expansion time, when the variable's value is read
- Overrides are written as `<VARIABLE>:<override> = "some_value"`
- A different syntax was used before **Honister** (3.4), with no retrocompatibility: `<VARIABLE>_<override> = "some_value"`



Overrides to modify variable values

- The `append` override adds **at the end** of the variable (without space).
  - `IMAGE_INSTALL:append = " dropbear"` adds `dropbear` to the packages installed on the image.
- The `prepend` override adds **at the beginning** of the variable (without space).
  - `PATH:prepend = "${COREBASE}/scripts/native-intercept:"` adds before the ones already present.
- The `remove` override removes all occurrences of a value within a variable.
  - `IMAGE_INSTALL:remove = "i2c-tools"`

Order of variable assignment



1. All the operators are applied, in parsing order
2. `:append` overrides are applied
3. `:prepend` overrides are applied
4. `:remove` overrides are applied



Overrides for conditional assignment

- Append the machine name to only define a configuration variable for a given machine.
- It tries to match with values from OVERRIDES which includes MACHINE, SOC_FAMILY, and more.
- If the override is in OVERRIDES, the assignment is applied, otherwise it is ignored.

```
OVERRIDES = "arm:armv7a:ti-soc:ti33x:beaglebone:poky"

KERNEL_DEVICETREE:beaglebone = "am335x-bone.dtb" # This is applied
KERNEL_DEVICETREE:dra7xx-evm = "dra7-evm.dtb"    # This is ignored
```



Overrides for conditional assignment: precedence

- The most specific assignment takes precedence.
- Example:

```
IMAGE_INSTALL:beaglebone = "busybox mtd-utils i2c-tools"
IMAGE_INSTALL = "busybox mtd-utils"
```

- If the machine is `beaglebone`:
  - `IMAGE_INSTALL = "busybox mtd-utils i2c-tools"`
- Otherwise:
  - `IMAGE_INSTALL = "busybox mtd-utils"`



Combining overrides

- The previous methods can be combined.
- If we define:
  - `IMAGE_INSTALL = "busybox mtd-utils"`
  - `IMAGE_INSTALL:append = " dropbear"`
  - `IMAGE_INSTALL:append:beaglebone = " i2c-tools"`
- The resulting configuration variable will be:
  - `IMAGE_INSTALL = "busybox mtd-utils dropbear i2c-tools"` if the machine being built is `beaglebone`.
  - `IMAGE_INSTALL = "busybox mtd-utils dropbear"` otherwise.



Virtual providers



Introduction to virtual providers

- Some recipes have the same purpose, and only one can be used at a time.
- The build system uses **virtual providers** to reflect this.
- Only one of the recipes that provides the functionality will be compiled and integrated into the resulting image.



Variant examples

- The virtual provider names are often in the form `virtual/<name>`
- Example of available virtual providers with some of their variants:
  - `virtual/bootloader`: u-boot, u-boot-ti-staging...
  - `virtual/kernel`: linux-yocto, linux-yocto-tiny, linux-yocto-rt, linux-ti-staging...
  - `virtual/libc`: glibc, musl, newlib
  - `virtual/xserver`: xserver-xorg



Provider selection

- Providers are selected thanks to the PREFERRED_PROVIDER configuration variable.
- The recipe names **have to** suffix this variable.
- Examples:
  - `PREFERRED_PROVIDER_virtual/kernel ?= "linux-ti-staging"`
  - `PREFERRED_PROVIDER_virtual/libgl = "mesa"`



Version selection

- By default, Bitbake will try to build the recipe with the highest version number, from the highest priority layer, unless the recipe defines `DEFAULT_PREFERENCE = "-1"`
- When multiple recipe versions are available, it is also possible to explicitly pick one with PREFERRED_VERSION.
- The recipe names **have to** suffix this variable.
- **%** can be used as a wildcard.
- Example:
  - `PREFERRED_VERSION_nginx = "1.20.1"`
  - `PREFERRED_VERSION_linux-yocto = "5.14%"`



Selection of packages to install



Selection of packages to install

- Building recipes will result in binary packages being generated.
- The set of packages installed into the image is defined by the target you choose (e.g. `core-image-minimal`).
- It is possible to have a custom set by defining our own target, and we will see this later.
- When developing or debugging, adding packages can be useful, without modifying the recipes.
- Packages are controlled by the IMAGE_INSTALL configuration variable.



The power of BitBake



Common BitBake options

- BitBake can be used to run a full build for a given target with `bitbake [target]`
  - `target` is a recipe name, possibly with modifiers, e.g. `-native`
  - `bitbake ncurses`
  - `bitbake ncurses-native`
- But it can be more precise, with additional options:

`-c <task>`
: execute the given task

`-s`
: list all available recipes and their versions

`-f`
: force the given task to be run by removing its stamp file

`world`
: keyword for all recipes



BitBake examples

- `bitbake -c listtasks virtual/kernel`
  - Gives a list of the available tasks for the recipe providing the package `virtual/kernel`. Tasks are prefixed with `do_`.
- `bitbake -c menuconfig virtual/kernel`
  - Execute the task `menuconfig` on the recipe providing the `virtual/kernel` package.
- `bitbake -f dropbear`
  - Force the `dropbear` recipe to run all tasks.
- `bitbake --runall=fetch core-image-minimal`
  - Download all recipe sources and their dependencies.
- For a full description: `bitbake --help`



shared state cache

- BitBake stores the output of each task in a directory, the shared state cache.
- This cache is used to speed up compilation.
- Its location is defined by the SSTATE_DIR variable and defaults to `build/sstate-cache`.
- Over time, as you compile more recipes, it can grow quite big. It is possible to clean old data with:

```
$ find sstate-cache/ -type f -atime +30 -delete
```

This removes all files that have last been accessed more than 30 days ago (for example).

Practical lab - Advanced Yocto configuration



- Modify the build configuration
- Customize the package selection
- Experiment with BitBake
- Mount the root file system over NFS